Che cos'è un paradigma

L01 · Stato, funzioni e relazioni attraverso esempi

Mauro Leoncini

Administrivia

- Titolo: *Paradigmi di programmazione*
- CFU/ore in aula/ore di lavoro: 6/48/150
- Docente: Mauro Leoncini
- Ricevimento: Lunedì 16-17, Mercoledì 11-12 (previo appuntamento)
- Esame: scritto (70%) e laboratorio (30%). Un **esempio completo**, che illustra fedelmente la struttura della prova scritta, sarà consegnato entro le prime due settimana di lezione. Orale su richiesta dello studente o del docente, con effetto in entrambe le direzioni, ma solo dopo aver superato le prove scritta e di laboratorio.
- Codice etico: la paternità di ogni idea/dimostrazione/programma va chiaramente dichiarata.

Perché studiare altri paradigmi programmativi?

Di più, perché mai studiare **oggi** qualsivoglia aspetto della programmazione?

Guardiamo in faccia la realtà: un sistema di AI generativa può ormai produrre programmi in qualsiasi linguaggio, **più velocemente** e, molto spesso, **meglio di un programmatore umano**.

Ci sono cose che possono rimanere nelle competenze del professionista umano? Proviamo ad elencarne qualcuna.

- Saper formulare con precisione il problema e indicare il **paradigma** più appropriato;
- essere in grado di stabilire le proprietà che il programma dovrà soddisfare;
- verificare se le proprietà richieste al programma sono soddisfatte, valutarne l'**efficienza**, suggerire miglioramenti;
- individuare e interpretare eventuali controesempi che violano i vincoli posti.
- consegnare al *committente* un prodotto finito, documentato, manutenibile, aggiornabile.

Un problema, tre modi di risolverlo

Useremo un problema molto semplice — la concatenazione di due liste — per mettere a confronto tre diversi **modelli di computazione**:

1. come trasformazione di uno stato → paradigma *imperativo*;
2. come valutazione di una funzione → paradigma *funzionale* (`OCaml`)
3. come soddisfacimento di una relazione → paradigma *logico* (`Prolog`)

Lo scopo non è semplicemente di imparare nuovi linguaggi.

Lo scopo è rendersi conto che, cambiando paradigma, cambia il modo in cui **formuliamo il problema e ragioniamo sulla sua soluzione**.

Il ciclo di lavoro che useremo

Il corso seguirà ripetutamente questo ciclo:

$$\text{specifica} \longrightarrow \text{generazione} \longrightarrow \text{verifica} \longrightarrow \text{controesempio} \longrightarrow \text{revisione}$$

La generazione può essere svolta da una persona, da uno strumento automatico o da un sistema di AI. Il punto è comprendere che la correttezza del risultato non deriva (solo) da chi fisicamente produce il codice.

Il nostro primo problema modello

Consideriamo due liste di numeri interi (rappresentate qui utilizzando una **sintassi** ragionevole ma arbitraria):

```
[1; 2]        [3; 4]
```

Vogliamo ottenere:

```
[1; 2; 3; 4]
```

Chiamiamo questa operazione **concatenazione**.

Non ci interessa una specifica **sintassi**. Ci interessa mettere a fuoco soltanto **come istruire** il calcolatore usando i diversi modelli (indipendentemente da **chi** formula le istruzioni: uomo o sistema di AI generativa).

Paradigma imperativo: dobbiamo descrivere la procedura

Gli incredienti tipici possono essere:

- Un'opportuna struttura dati che conterrà il risultato della concatenazione
- L'inizializzazione della struttura con una **copia** gli elementi della prima lista
- Un ciclo per "scansionare" gli elementi della seconda lista
- L'aggiunta dell'elemento scansionato in coda alla risultato in costruzione

Il programma dice **come** concatenare. Lo stato — la struttura e l'elemento correntemente scansionato — è la parte che si deve gestire a mano, ed è anche quella in cui più facilmente si possono commettere errori.

Paradigma (dichiarativo) funzionale: si descrive la funzione

Attenzione però alla sostanza! La funzione non è giusto un nome diverso per la procedura imperativa.

Una definizione ricorsiva può essere vista piuttosto come un **sistema di equazioni funzionali** che caratterizza la funzione mediante uno o più casi base e una relazione ricorsiva.

- Dunque niente array, niente ciclo: la ricorsione porta con sé lo stato
- Il programma dice **che cosa** si calcola, in funzione di che cosa.
- Fissata anche in questo caso una sintassi, esprimiamo situazioni alternative relative al concatenamento di due liste `xs` e `ys`:
  1. il caso di lista `xs` vuota,
  2. il caso di lista `xs` con almeno un elemento.

Paradigma (dichiarativo) logico: si descrive la relazione

- Il programma non dice neppure che cosa si calcola, enuncia solo **fatti e regole** che riguardano le liste
- Un solo fatto, che afferma qual è il risultato del concatenamento di una lista vuota con una seconda lista `ys`
- Una sola regola, in base alla quale **dedurre qualcosa** dal fatto di sapere che il concatenamento delle liste `xs` e `ys` è una lista `zs`
- Anche in questo caso dovremo **scegliere una sintassi**.

Tre programmi corrispondenti: C++

```
std::list<int> concat(const std::list<int>& xs, const std::list<int>& ys) {
    std::list<int> result = xs;
    result.insert(result.end(), ys.begin(), ys.end());
    return result;
}
```

- La struttura scelta è `std::list`, in cui viene inizialmente copiata `xs`
- La "scansione" delle liste nel C++ moderno avviene mediante specifici *iteratori*
- Il metodo `insert`, proprio di `std::list`, inserisce elementi in un punto della lista specificato dal primo argomento. Gli altri due argomenti (sempre iteratori) permettono di specificare una "spezzone" di elementi contigui di una seconda lista da inserire (in questo caso tutta la seconda lista)

Stato

In un programma imperativo una variabile è spesso vista come un **contenitore**: ciò che ci sta dentro può cambiare.

```
result = [1; 2]
result = [1; 2; 3]
result = [1; 2; 3; 4]
```

Il nome `result` rimane lo stesso, mentre il valore associato cambia nel tempo.

Lo **stato** di una computazione raccoglie le informazioni memorizzate in un certo istante. Un comando trasforma uno stato in un altro stato.

$$S_0 \longrightarrow S_1 \longrightarrow S_2 \longrightarrow \cdots$$

Domande nascoste nella soluzione imperativa

L'apparente semplicità della soluzione nasconde diverse decisioni:

- che cosa significa esattamente che «`xs` viene copiata in `result`»?
- `result` è cioè una nuova lista oppure un altro nome per `xs` (**aliasing**)?
- come viene eseguita l'aggiunta di un elemento (in fondo) a `std::list`? E, più in generale, in un punto qualsiasi della lista?
- `xs` oppure `ys` possono cambiare durante il calcolo?
- chi osserva le liste dall'esterno può accorgersi delle modifiche?

Lo stato è uno strumento potente, ma introduce anche problemi di identità, condivisione e ordine temporale delle operazioni.

Tre programmi corrispondenti: OCaml

```
let rec append((xs : int list), (ys : int list)) =
if xs = [] then
  ys
else
  List.hd xs :: append(List.tl xs,ys)

let () =
  List.iter (fun x -> Printf.printf "%d " x) (append([1; 2], [3; 4]));
  print_newline ()
```

- La lista vuota in OCaml si indica con []
- Se una lista `xs` non è vuota, allora è fatta di un prima elemento, `List.hd x`, e dei rimanenti elementi `List.tl x`, che costituiscono a loro volta una lista
- Un elemento `e` e una lista `xs` formano una lista `e :: xs` in cui `e` è il primo elemento e `xs` il resto della lista
- Si faccia ben conto che la struttura **if then else** non è un meccanismo di controllo come in C++; qui definisce una sorta di **sistema di equazioni** la cui soluzione è la funzione di concatenazione

---

Come procede la valutazione?

- In un linguaggio funzionale gli oggetti sono **immutabili**
- Ad ogni passaggio viene creata una lista nuova

`append ([1; 2] [3; 4])`

OCaml procede nel modo seguente

```
append ([1; 2] [3; 4])
= 1 :: append ([2] [3; 4])
= 1 :: (2 :: append ([] [3; 4]))
= 1 :: (2 :: [3; 4])
= 1 :: [2; 3; 4]
= [1; 2; 3; 4]
```

Tre programmi corrispondenti: Prolog

```
append([], Ys, Ys).

append([X|Xs], Ys, [X|Zs]) :-
    append(Xs, Ys, Zs).
```

Le due *clausole* (un **fatto** e una **regola**) hanno la seguente lettura dichiarativa (già accennata), che permette di definire completamente la relazione *append*:

1. la concatenazione della lista vuota e di `Ys` è `Ys`;
2. se `Zs` è la concatenazione di `Xs` e `Ys`, allora `[X|Zs]` è la concatenazione di `[X|Xs]` e `Ys`.

Le iniziali maiuscole indicano variabili.

Una query nella direzione consueta

Ok, ma noi vogliamo determinare la concatenazione di due liste ben precise? Come procediamo?

Entra in gioco il terzo tipo di **statement** (oltre a fatti e regole), ovvero la **query**

```
?- append([1,2], [3,4], Zs).
```

Risposta:

```
Zs = [1,2,3,4].
```

Abbiamo fornito le prime due liste e chiesto la terza. Prolog ha "trovato" una lista che soddisfa i fatti e le regole note

---

Che cosa ha fatto Prolog?

Per ora ci basta una descrizione intuitiva.

Prolog:

- confronta la query con le clausole del programma;
- cerca sostituzioni per le variabili che rendano compatibili i termini;
- esplora le possibili alternative;
- restituisce le sostituzioni che rendono vera la query.

Studieremo in seguito i meccanismi precisi:

- unificazione;
- risoluzione;

- ricerca;
- backtracking.

Una relazione non ha una sola direzione

La cosa veramente interessante è che non esiste una sola **direzione** del calcolo!

```
?- append(Xs, Ys, [1,2,3]).
```

Prolog trova progressivamente:

```
Xs = [],        Ys = [1,2,3] ;
Xs = [1],       Ys = [2,3]   ;
Xs = [1,2],     Ys = [3]     ;
Xs = [1,2,3],   Ys = []      ;
false.
```

La stessa definizione viene quindi usata per scomporre una lista in tutti i modi possibili.

Un'altra direzione ancora

```
?- append([1,2], Ys, [1,2,3,4]).
```

Risposta:

```
Ys = [3,4].
```

Non abbiamo chiesto al programma di «rimuovere un prefisso». Abbiamo posto un problema relazionale:

> Per quale valore di `Ys` è vera la relazione richiesta?

Una relazione può quindi essere utilizzata fornendo informazioni diverse nelle diverse query.

Una prima domanda dovrebbe ora sorgere spontanea

> Che cosa significa dunque **programmare**?

Una possibile risposta è:

> Programmare consiste nel descrivere computazioni in una forma che possa essere eseguita automaticamente.

Questa risposta lascia però aperta la questione più interessante:

> Quali concetti possiamo utilizzare per descrivere la computazione?

Dipende dal paradigma. Possiamo parlare di **comandi** che modificano la memoria (lo **stato**), di **funzioni** che producono valori oppure di **relazioni** che devono risultare vere.

Che cos'è un paradigma di programmazione?

Un **paradigma di programmazione** è un modo di rappresentare e organizzare una computazione. Stabilisce quali entità consideriamo fondamentali e quali forme di ragionamento risultano naturali.

Paradigma	Entità centrali	Il programma descrive in primis
Imperativo	Variabili, memoria, comandi	Come trasformare uno stato
Funzionale	Valori, espressioni, funzioni, equazioni funzionali	Come ottenere un valore da altri valori
Logico	Termini, relazioni, vincoli	Quali condizioni devono essere vere

Due programmi possono assomigliarsi **sintatticamente** ma adottare modelli di computazione profondamente diversi.

La concatenazione nei tre paradigmi

Imperativo	Funzionale	Logico
Costruisce e modifica un risultato	Definisce il valore mediante equazioni	Definisce quando una relazione è vera
Sequenza di comandi	Valutazione di espressioni	Ricerca di sostituzioni
Stato esplicito	Valori immutabili	Variabili logiche
Direzione di calcolo fissata	Argomenti e risultato fissati dalla funzione	Possibili modalità d'uso diverse

---

Linguaggio e paradigma non sono sinonimi

Un linguaggio concreto può supportare più stili di programmazione:

- in OCaml si possono usare funzioni pure, ma anche strutture modificabili ed effetti;
- molti linguaggi imperativi permettono di usare funzioni di ordine superiore;
- Prolog possiede un significato dichiarativo, ma l'ordine delle clausole influenza la ricerca delle soluzioni.

Nel corso useremo soprattutto:

- **OCaml**, per studiare la programmazione funzionale;
- **Prolog**, per studiare la programmazione logica.

Parleremo quindi di paradigmi come **modelli concettuali**, non come compartimenti assolutamente separati.

Un secondo esempio modello

Dato un grafo orientato, stabilire se da un nodo se ne raggiunge un altro (problema ben noto in algoritmica con il nome di **graph reachability**)

- Ancora una volta, prima di presentare i programmi nei rispettivi linguaggi (la **sintassi**), dobbiamo capire **che cosa bisogna dire** al calcolatore in ciascuno dei tre casi.

Proiettare i tre file di codice/ affiancati, non incollarli nelle slide: si vuole che vedano la differenza di lunghezza prima di leggere una riga. C++ circa 30 righe, OCaml 14, Prolog 6 più i commenti.

Paradigma imperativo: si descrive la procedura

- una pila dei nodi da visitare
- un insieme dei nodi già visitati
- un ciclo che consuma la pila

Il programma dice **come** effettuare le ricerca.

Lo stato — pila e insieme — è la parte che si deve gestire a mano (la più rischiosa).

Paradigma funzionale: si descrive la funzione

- niente pila, niente ciclo: la ricorsione porta con sé lo stato
- l'elenco dei visitati resta, ma diventa un **argomento** della funzione

Il programma dice **che cosa** si calcola, in funzione di che cosa.

Paradigma logico: si descrive la relazione

Dobbiamo solo scrivere relazioni che siano soddisfatte da nodi collegati da un cammino

1. Esiste sempre un cammino da un nodo X a se stesso
2. Se X e Y sono nodi diversi, un cammino da X a Y è implicato dall'esistenza di un arco (X,Z) e da un cammino da Z a Y

Non si dice **come** effettuare la ricerca: la strategia sul modo di procedere è del linguaggio, non del programma.

Graph reachability: C++

```
using Grafo = std::map<std::string, std::vector<std::string>>;

const Grafo archi = {
    {"a", {"b"}},
    {"b", {"d"}},
    {"c", {"a", "b"}},
    {"d", {}}
};
```

Graph reachability: C++ (2)

```cpp
bool raggiungibile(const std::string& da, const std::string& a) {
    std::stack<std::string> daVisitare;
    std::set<std::string> visitati;
    daVisitare.push(da);

    while (!daVisitare.empty()) {
        std::string nodo = daVisitare.top();
        daVisitare.pop();
        if (nodo == a) return true;
        if (!visitati.insert(nodo).second) continue;

        auto it = archi.find(nodo);
        if (it != archi.end())
            for (const std::string& vicino : it->second)
                daVisitare.push(vicino);
    }
    return false;
}
```

Graph reachability: OCaml

```ocaml
let archi = [ ("a", [ "b" ]); ("b", [ "d" ]);
              ("c", ["a"; "b"]); ("d", []) ]

let vicini n = try List.assoc n archi with Not_found -> []

let rec raggiungibile visitati da a =
  da = a
  || (not (List.mem da visitati)
      && List.exists (fun v -> raggiungibile (da :: visitati) v a)
                     (vicini da)
     )
```

Graph reachability: Prolog

```
arco(a, b).
arco(b, d).
arco(c, a).
arco(c, b).

cammino(X, X).
cammino(X, Y) :- arco(X, Z), cammino(Z, Y).
```

Una query

```
?- cammino(c,d).
yes
```

La domanda che cambia tutto

```
?- cammino(a, X).    % quali nodi raggiungo da a
?- cammino(X, c).    % da quali nodi raggiungo c
```

Nessun argomento è l'ingresso e nessuno è l'uscita.

In C++ e in OCaml la seconda domanda richiede **un diverso programma**.

Qui si ferma la demo e si lascia la domanda aperta. È il seme di L13.

Il conto da pagare

Aggiungete `arco(d, a).` — un ciclo — e la query non termina più.

- il programma è ancora **logicamente corretto**
- ed è diventato **operativamente inservibile**

Queste due letture di uno stesso programma non coincidono. Tenerle distinte è il filo di tutto il corso; ci torniamo in una lezione molto più avanti.

Che cosa portiamo a casa oggi?

- Un paradigma non è una sintassi: è che cosa siete obbligati a dire all'esecutore
- Lo stato è una qualcosa che si può **gestire**, **passare** o **non avere**
- Un programma ha un **significato** e ha un **comportamento**, e non sono lo stesso

---